



Programmation Fonctionnelle

Projet Fourmilière - Akka / Scala 2

Rapport

[ING2 GSI2]

SWICZKA Adam

XU Hénoc

NICOLAO Hyacinthe

BADAoui Lina

KIM Aline

Professeure encadrante : Zaouche Djaouida

Soumis le 14 avril 2026

Sommaire

1. Vérification formelle pour les systèmes critiques (p.4)
 - 1.A) Vérification formelle (p.4)
 - 1.B) Model checking (p.4)
 - 1.C) Lien avec les tests unitaires (p.5)
2. Les réseaux de Pétri : modélisation et analyse de la concurrence (p.6)
 - 2.A) Formalisation de base (p.6)
 - 2.B) Traduction Akka $\rightarrow$ réseau de Pétri (p.6)
 - 2.C) Marquage initiale M_0 (p.8)
 - 2.D) Schéma du réseau de Pétri (p.9)
3. Propriétés structurelles analysables (p.10)
 - 3.A) Bornitude (p.10)
 - 3.B) P-invariants (p.10)
 - 3.C) T-invariants (p.11)
 - 3.D) Vivacité et absence de deadlock (p.12)
4. Propriétés LTL : formalisation sur les traces d'exécution (p.13)
 - 4.A) Propriétés de sûreté ($\square P$) (p.13)
 - 4.B) Propriétés de vivacité ($\Diamond P$ depuis tout état) (p.14)
 - 4.C) Terminaison et deadlocks (p.14)
5. Architecture Akka et modélisation fonctionnelle (p.16)
 - 5.A) Vue d'ensemble du système (p.16)
 - 5.B) Flux de messages critiques (p.16)
 - 5.C) Invariants métier et leur implémentation (p.17)
 - 5.D) Supervision et tolérance aux pannes (ColonyGuardianActor) (p.17)
 - 5.E) Pattern MessageAdapter (SimulationMonitor) (p.18)
6. Tests unitaires (p.19)
 - 6.A) Organisation (p.19)
 - 6.B) Tests des acteurs (ActorSpec) (p.19)
 - 6.C) Tests du réseau de Pétri (PetriNetSpec) (p.19)
 - 6.D) Lien tests $\leftrightarrow$ vérification formelle (p.20)

- 7. Simulation comparée : comportement réel vs modèle formel (p.21)
 - 7.A) Résultats de l'analyseur du réseau de Pétri (p.21)
 - 7.B) Observations de la simulation Akka (p.21)
 - 7.C) Limites du modèle formel et pistes d'amélioration (p.22)
- 8. Sources bibliographiques (p.23)
 - 8.A) Réseaux de Pétri — fondements théoriques (p.23)
 - 8.B) Vérification formelle et model checking (p.23)
 - 8.C) Modèle d'acteurs et Akka (p.23)
 - 8.D) Scala fonctionnel (p.23)

1. Vérification formelle pour les systèmes critiques

1.A) Vérification formelle

Un système distribué critique comme notre colonie de fourmis présente des caractéristiques qui rendent le test classique insuffisant : l'entrelacement non-déterministe des messages Akka, les états internes invisibles depuis l'extérieur, et des propriétés globales (la reine ne doit jamais mourir de faim si des transporteuses sont actives) qui ne peuvent pas être vérifiées par quelques exécutions.

La vérification formelle consiste à prouver mathématiquement qu'un système satisfait une propriété pour toutes les exécutions possibles, pas seulement celles observées lors des tests. Elle répond à deux questions distinctes :

- Est-ce que quelque chose de mauvais ne peut jamais arriver ? (propriétés de sûreté)
- Est-ce que quelque chose de bon finit toujours par arriver ? (propriétés de vivacité)

1.B) Model checking

Le model checking est la technique la plus répandue pour les systèmes concurrents. Le principe : on représente le système comme un graphe d'états (tous les états possibles + toutes les transitions possibles), puis on vérifie qu'une propriété est satisfaite dans chaque état du graphe. Si ce n'est pas le cas, le model checker produit un contre-exemple (un chemin d'exécution menant à la violation).

Le projet implémente son propre explorateur dans *petri.scala* (branche *main*), via la méthode *reachabilityGraph* qui effectue un BFS sur les marquages accessibles, limitée à *maxStates* pour contrôler l'explosion combinatoire.

Le défi central du model checking est l'explosion de l'espace d'états : le nombre de marquages accessibles croît exponentiellement avec le nombre de composants. Avec 4 fourrageuses (chacune dans 4 états possibles), 4 transporteuses (5 états), un stock de 0 à 20, et une reine (3 états), l'espace d'états théorique dépasse 10 millions de configurations. La limite à 2000 marquages dans *reachabilityGraph(2000)* est une approximation suffisante pour détecter les deadlocks courants et vérifier les propriétés LTL1–LTL9.

1.C) Lien avec les tests unitaires

Les tests unitaires de la branche final se répartissent en deux fichiers distincts : *ActorSpec.scala* couvre les acteurs Akka (*StorageActor*, *QueenActor*, *Invariants*, *EggActor* : soit 29 tests au total), et *PetriNetSpec.scala* couvre exclusivement le réseau de Pétri (structure, franchissabilité, P-invariants, BFS, LTL1–LTL9 : 12 groupes de tests).

Ces tests vérifient des invariants dans des exécutions particulières. Ils sont nécessaires mais non suffisants : un test peut passer sur 1000 exécutions et manquer un entrelacement rare qui viole l'invariant. La vérification formelle par réseau de Pétri est complémentaire : elle prouve les propriétés pour toutes les exécutions possibles, y compris les plus défavorables.

2. Les réseaux de Pétri : modélisation et analyse de la concurrence

2.A) Formalisme de base

Un réseau de Pétri est un graphe biparti (P, T, F) où :

- P est un ensemble fini de places : chaque place représente une condition, un état local, ou une ressource
- T est un ensemble fini de transitions : chaque transition représente un événement ou une action
- $F \subseteq (P \times T) \cup (T \times P)$ est l'ensemble des arcs, chaque arc portant un poids (entier positif)
- $M : P \rightarrow \mathbb{N}$ est le marquage : le nombre de jetons dans chaque place à un instant donné

La dynamique est régie par la règle de franchissement : une transition t est franchissable dans le marquage M si pour chaque place p en entrée de t , $M(p) \geq \text{poids}(p, t)$. Franchir t produit le marquage M' en soustrayant les jetons consommés et en ajoutant les jetons produits. C'est exactement ce qu'implémente *Marking.fire(t)* dans *petri.scala*.

Dans notre projet, les jetons représentent des unités de nourriture dans le stockage ($M(\text{stock}) \in [0, 20]$), des fourmis dans un état donné (1 jeton dans *forager_idle* = 1 fourrageuse disponible), ou des unités de faim de la reine (abstraites par une garde).

2.B) Traduction Akka → réseau de Pétri

Le choix du modèle d'acteurs Akka facilite la traduction vers les réseaux de Pétri car les deux partagent la même intuition : des entités locales qui communiquent par échange de messages. La correspondance systématique établie dans *petri.scala* est la suivante :

Chaque état comportemental d'un acteur (les différents *Behaviors.receive* dans les FSM) devient une place. Ainsi *ForagerAntActor* génère les places *forager_idle*, *forager_resting*, *forager_resting_waiting* et *forager_dead*. Un jeton dans l'une de ces places signifie qu'une fourrageuse est dans cet état.

Chaque réception de message déclenchant un changement d'état devient une transition. La réception de *SearchFood* en état idle devient $t_{\text{search_food}}$: elle consomme 1 jeton de *forager_idle* et en produit 1 dans *forager_resting* ainsi que 2 dans *stock* (dépôt de nourriture, poids moyen).

Les conditions de garde dans le code Scala (*if hunger > MaxHunger, if starvation >= 3*) se traduisent par des arcs inhibiteurs ou des gardes booléennes sur les transitions. *t_queen_dies* est franchissable seulement si *hunger >= MaxHunger*, tandis que *t_tick* porte la garde *hunger < MaxHunger* (les deux sont mutuellement exclusives).

Les ressources partagées (le stockage) deviennent des places bornées. Le *StorageActor* borné à 20 se traduit par la place *stock* dont le marquage est contraint : $M(stock) \in [0, 20]$.

Quant au *ColonyGuardianActor*, il constitue le superviseur racine de la colonie, ajouté dans la version finale du projet. Cet acteur ne correspond pas directement à des places dans le réseau de Pétri car il opère à un niveau méta : il définit des stratégies de tolérance aux pannes via un protocole interne (*GuardianCommand*) et non des messages métier. Sa contribution au modèle formel est indirecte, il garantit que les acteurs critiques (*StorageActor*, *QueenActor*) sont redémarrés ou arrêtés correctement en cas d'exception, ce qui renforce la validité des invariants LTL dans les scénarios de défaillance.

Acteur Akka	Stratégie de supervision	Justification métier
StorageActor	restart (max 3 fois / 10s)	Acteur critique : repart à stock=0 plutôt que de faire tomber la colonie
QueenActor	stop	Mort = fin de la colonie : pas de redémarrage possible
ForagerAntActor	restartWithBackoff (500ms-5s)	Peut replanter et reprendre en état idle
CarrierAntActor	restartWithBackoff (500ms-5s)	Même raisonnement que la fourrageuse

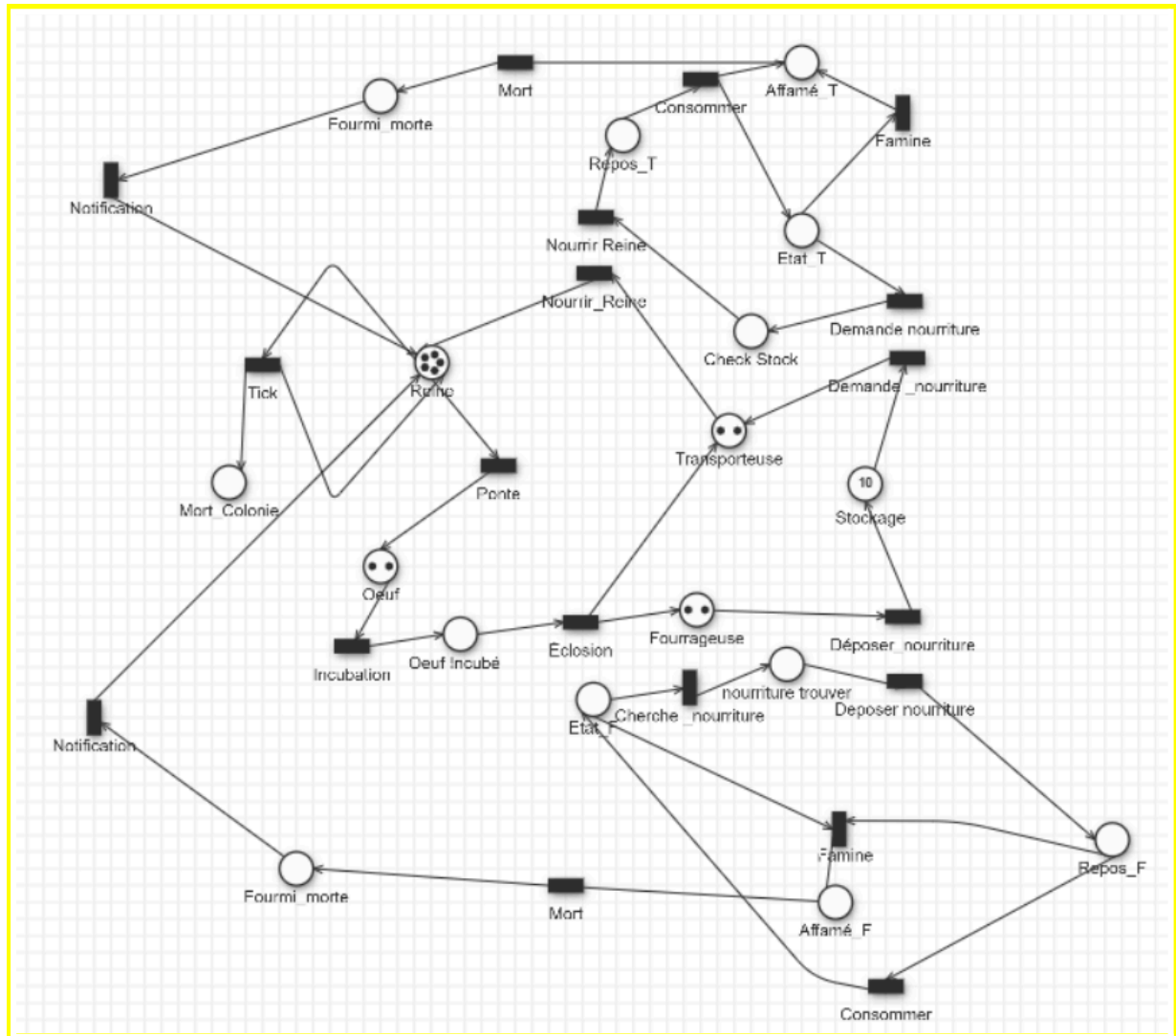
2.C) Marquage initial M_0

M_0 correspond directement à l'état au démarrage de *Simulation.scala* : reine vivante (*hunger* = 5), 1 fourrageuse idle, 1 transporteuse idle, stock vide. Dans *petri.scala* :

```
535   val initialMarking: Marking = Marking(Map(  
536     queenAlive -> 1,  
537     hungerToken -> 5,    // initialHunger = 5  
538     foragerIdle -> 1,  
539     carrierIdle -> 1  
540     // stock = 0, eggCycle1 = 0, eggCycle2 = 0 (implicites)  
541   ))
```

Le paramètre *hunger* = 5 est représenté par 5 jetons dans *hungerToken*. C'est l'une des spécificités du modèle : contrairement aux versions précédentes où la faim était abstraite comme une garde extérieure, la version finale encode directement les niveaux de faim sous forme de jetons dans une place dédiée, ce qui permet une vérification formelle plus précise des propriétés LTL4 et LTL9.

2.D) Schéma du réseau de Pétri



3. Propriétés structurelles analysables

3.A) Bornitude

Un réseau est k-borné si pour tout marquage atteignable M et toute place p , $M(p) \leq k$. Une place non bornée signale un risque de débordement (accumulation infinie de jetons).

Dans notre réseau, deux places sont bornées par construction métier : *stock* est borné à 20 (via le min *MaxCapacity* dans *StorageActor*), et les places de population (*forager_idle* + *forager_resting* + *forager_resting_waiting* + *forager_dead* ≤ 4) sont bornées par les quotas. La méthode *deadlocks* de *PetriNet* vérifie indirectement la bornitude en explorant *reachabilityGraph* : si l'exploration termine, le réseau est borné dans la limite des marquages explorés.

3.B) P-invariants

Un P-invariant est un vecteur $y \in \mathbb{Z}^{|P|}$ tel que pour tout franchissement, la quantité $\sum y(p) * M(p)$ reste constante. Formellement : $y^T * C = 0$ où $C = Post - Pre$ est la matrice d'incidence.

Six P-invariants sont identifiés et vérifiés dans *checkPInvariants* de *petri.scala* :

PI1 : Conservation de la reine.

$M(\text{queen_alive}) + M(\text{queen_dead}) + M(\text{queen_sated}) = 1$. La reine est toujours dans exactement un état. Cela traduit formellement l'invariant [I7] de *Invariants.scala* : *queenIsAlive* = la faim est strictement sous *MaxHunger*.

PI2 : Borne de faim.

$0 \leq M(\text{hunger_token}) \leq 10$. Traduit les invariants [I1] (faim ≥ 0) et [I2] (faim $\leq \text{MaxHunger}$).

PI3 : Conservation des fourrageuses.

$M(\text{forager_idle}) + M(\text{forager_resting}) + M(\text{forager_resting_waiting}) + M(\text{forager_dead}) \in [0, 4]$. Une fourrageuse est toujours dans exactement un état, et le quota de 4 est respecté. Cela traduit l'invariant [I5].

PI4 : Conservation des transporteuses.

Analogue pour *carrier_idle* + *carrier_waiting* + *carrier_resting* + *carrier_resting_waiting* + *carrier_dead* ≤ 4 . Traduit l'invariant [I6].

PI5 : Borne du stock.

$0 \leq M(stock) \leq 20$. Correspond aux invariants [I3] et [I4] de *Invariants.scala*, vérifiés à chaque modification dans *StorageActor*.

PI6 : Cohérence vivacité reine.

$M(queen_alive) = 1 \rightarrow M(hunger_token) \leq 10$. Traduit l'invariant [I7] : si la reine est vivante, sa faim reste bornée.

Ces P-invariants sont structurels : ils sont vrais pour tout marquage atteignable, indépendamment des valeurs initiales, du moment où on les vérifie, et de l'entrelacement des messages. La vérification analytique ($y^A T * C = 0$) est implémentée dans *isPInvariant* et confirmée par *PetriNetSpec* pour P11.

3.C) T-invariants

Un T-invariant est un vecteur $x \in \mathbb{N}^{|T|}$ tel que $C * x = 0$: une séquence de franchissements ramenant au marquage initial. Les T-invariants représentent les cycles de comportement du système.

Quatre T-invariants principaux sont identifiés dans *petri.scala* :

T-invariant	Transitions	Description
TI1	t_search_food t_forager_consume t_forager_fed	Cycle nominal fourrageuse : collecte → nourrissage → retour idle
TI2	t_request_food t_carrier_deliver t_carrier_consume t_carrier_fed	Cycle nominal transporteuse : demande stock → livraison reine → repos → retour idle
TI3	t_tick t_carrier_deliver	Cycle faim reine : tick augmente hunger, livraison le réduit
TI4	t_lay_egg t_egg_incubate t_spawn_forager	Cycle de vie d'un oeuf : ponte → incubation (2 cycles) → naissance fourrageuse

L'existence de TI3 garantit qu'il existe au moins une séquence d'exécution dans laquelle la colonie survit indéfiniment : condition nécessaire (mais non suffisante) pour la vivacité. TI4 confirme que le cycle de reproduction est cohérent avec le modèle en deux places distinctes (*egg_cycle1*, *egg_cycle2*) corrigé dans la version finale.

3.D) Vivacité et absence de deadlock

Une transition est vivante si elle est franchissable dans tout futur accessible depuis tout marquage atteignable. Une transition morte signale un comportement inaccessible. Par exemple, si `t_search_food` est morte, les fourrageuses ne cherchent jamais de nourriture.

Un deadlock est un marquage où aucune transition n'est franchissable. Dans *petri.scala*, *deadlocks()* filtre les marquages de *reachabilityGraph* où *enabled(m).isEmpty*. La propriété LTL8 formalisée dans le code exprime cela directement : $\square (deadlock \rightarrow M(queen_dead) = 1)$. Un deadlock n'est acceptable que si la reine est morte, ce qui est l'état terminal intentionnel du système.

4. Propriétés LTL : formalisation sur les traces d'exécution

La Linear Temporal Logic (LTL) est une logique modale dont les formules s'évaluent sur des traces, des séquences infinies d'états. Elle permet d'exprimer des propriétés temporelles sur les exécutions du système, en complément des propriétés structurelles (P/T-invariants) qui ne dépendent pas du temps.

Les quatre opérateurs fondamentaux sont $\Box$ (always : dans tous les états futurs), $\Diamond$ (eventually : dans au moins un état futur), U (until : ϕ reste vrai jusqu'à ce que ψ devienne vrai), et $\circ$ (next : vrai dans l'état immédiatement suivant).

Dans *petri.scala*, neuf propriétés LTL sont formalisées et vérifiées par exploration BFS de l'espace d'états (méthodes *verifyLTL1* à *verifyLTL9*). Toutes retournent *true* sur l'espace exploré.

4.A) Propriétés de sûreté ($\Box P$)

LTL1 : Sûreté de l'état de la reine.

$\Box (M(\text{queen_alive}) + M(\text{queen_dead}) + M(\text{queen_sated}) = 1)$. La reine est toujours dans un état bien défini. Correspond au P-invariant PI1 et à l'invariant métier [I7].

LTL2 : Sûreté du stock.

$\Box (M(\text{stock}) \geq 0)$. Le stock ne devient jamais négatif. C'est le P-invariant PI5, correspondant à l'invariant [I3] de *Invariants.scala* vérifié par *checkStorage*. Garanti structurellement : le stockage ne livre que ce qu'il a.

LTL3 : Sûreté de la population.

$\Box (\text{population}(F) \leq 4 \wedge \text{population}(C) \leq 4)$. La population des fourmis est toujours bornée. Correspond aux P-invariants PI3 et PI4, et aux invariants [I5] et [I6].

LTL4 : Sûreté de la faim.

$\Box (M(\text{queen_alive}) = 1 \rightarrow 0 \leq M(\text{hunger_token}) \leq 10)$. La faim reste bornée si la reine est vivante. Correspond aux P-invariants PI2 et PI6 et aux invariants [I1], [I2].

4.B) Propriétés de vivacité ($\Diamond P$ depuis tout état)

LTL5 : Vivacité des fourrageuses.

$\square (M(\text{forager_idle}) \geq 1 \rightarrow \Diamond t_search_food \text{ franchissable})$. Une fourrageuse idle finit toujours par chercher de la nourriture. Garantie par les schedulers internes (`scheduleAtFixedRate(6.seconds)` dans `ForagerAntActor`), et vérifiée formellement dans le réseau : `t_search_food` ne dépend que de `forager_idle` et est donc immédiatement franchissable si ce jeton est présent.

LTL6 : Vivacité des transporteuses.

$\square (M(\text{carrier_idle}) \geq 1 \wedge M(\text{stock}) \geq 2 \rightarrow \Diamond t_carrier_deliver \text{ franchissable})$. Toute transporteuse avec stock disponible finit par livrer. Conditionnelle à la disponibilité du stock.

LTL7 : Vivacité des œufs.

$\square (M(\text{egg_cycle1}) \geq 1 \rightarrow \Diamond t_egg_incubate \text{ franchissable})$. Tout œuf en cycle 1 progresse vers le cycle 2. Vérifiable immédiatement : `t_egg_incubate` ne requiert que 1 jeton dans `egg_cycle1`.

4.C) Terminaison et deadlocks

LTL8 : Non-deadlock vivant.

$\square (\text{deadlock} \rightarrow M(\text{queen_dead}) = 1)$. Le seul état terminal acceptable est la mort de la reine. C'est la propriété la plus importante pour la correction du système. Vérifiée par `verifyLTL8(deadlockMarkings)` sur l'ensemble des marquages sans transition franchissable.

LTL9 : Terminaison inévitable.

$\square ((\text{fourmis_actives} = 0 \wedge M(\text{stock}) = 0) \rightarrow \Diamond M(\text{queen_dead}) = 1)$. Sans fourmis ni stock, la reine meurt inévitablement. Vérifiable par exploration : dans tout marquage où les places de fourmis actives sont vides et le stock nul, `t_tick` reste franchissable et les `hunger_token` s'accumulent jusqu'à `t_queen_dies`.

Propriété LTL	Formule
LTL1	$\square$ (état reine unique)
LTL2	$\square$ (stock ≥ 0)
LTL3	$\square$ (population ≤ 4)
LTL4	$\square$ (reine vivante $\rightarrow$ faim ≤ 10)

LTL5	$\square (\text{forager_idle} \geq 1 \rightarrow \Diamond t_search_food)$
LTL6	$\square (\text{carrier_idle} \geq 1 \wedge \text{stock} \geq 2 \rightarrow \Diamond t_carrier_deliver)$
LTL7	$\square (\text{egg_cycle1} \geq 1 \rightarrow \Diamond t_egg_incubate)$
LTL8	$\square (\text{deadlock} \rightarrow \text{queen_dead})$
LTL9	$\square (\text{fourmis} = 0 \wedge \text{stock} = 0 \rightarrow \Diamond \text{queen_dead})$

5. Architecture Akka et modélisation fonctionnelle

5.A) Vue d'ensemble du système

Le système simule la survie d'une colonie de fourmis autour d'un invariant critique : la Reine ne doit pas mourir de faim (faim maximale = 10). Chaque acteur est une FSM (machine à états finis) qui communique exclusivement par messages asynchrones Akka Typed.

Acteur Rôle	États	FSM
QueenActor	Acteur central. Gère la faim, pond des œufs, supervise naissances/morts	Actif / Mort (Behaviors.stopped)
EggActor	Incubation sur 2 Ticks. Éclot en Fourrageuse ou Transporteuse (aléatoire)	incubating
ForagerAntActor	Cherche de la nourriture, la dépose au stockage	idle → resting → restingWaiting
CarrierAntActor	Prend du stockage, livre à la reine	idle → waiting → resting → restingWaiting
StorageActor	Buffer central borné à 20 unités. Sériailise tous les accès concurrents	Stateless (stock: Int)
ColonyGuardianActor	Superviseur racine. Définit les stratégies de tolérance aux pannes	waiting
SimulationMonitor	Observe la reine via watch(). Illustre le pattern MessageAdapter	Actif / Arrêté

5.B) Flux de messages critiques

Les interactions entre acteurs suivent un flux déterministe qui assure la survie de la reine :

- Fourrageuse —DepositFood— → Stockage
- Transporteuse —RequestFood— → Stockage —FoodReady— → Transporteuse
- Transporteuse —FeedQueen— → Reine
- Fourrageuse/Transporteuse (repos) —ConsumeFood— → Stockage
- Œuf —SpawnAnt— → Reine —spawn— → Nouvelle fourmi
- Fourmi morte —AntDied— → Reine
- Simulation —Tick (toutes les 4s)— → Reine

5.C) Invariants métier et leur implémentation

Les invariants métier sont centralisés dans *Invariants.scala* et vérifiés à chaque événement critique dans les acteurs. Les sept invariants définis sont :

Invariant	Formule	Vérifié dans	Test unitaire
I1	$\text{hunger} \geq 0$	QueenActor (Tick)	ActorSpec - QueenActor
I2	$\text{hunger} \leq \text{MaxHunger}=10$	QueenActor (Tick)	ActorSpec - QueenActor
I3	$\text{stock} \geq 0$	Structural (StorageActor)	ActorSpec - StorageActor
I4	$\text{stock} \leq \text{MaxCapacity}=20$	StorageActor (DepositFood)	ActorSpec - StorageActor
I5	$\text{foragerCount} \in [0, 4]$	QueenActor (SpawnAnt/AntDied)	ActorSpec - Invariants
I6	$\text{carrierCount} \in [0, 4]$	QueenActor (SpawnAnt/AntDied)	ActorSpec - Invariants
I7	$\text{queenIsAlive} \leftrightarrow \text{hunger} < 10$	QueenActor (Tick)	ActorSpec - Invariants

5.D) Supervision et tolérance aux pannes (ColonyGuardianActor)

La version finale du projet introduit le *ColonyGuardianActor*, un acteur superviseur racine qui enveloppe chaque acteur métier dans une stratégie *Behaviors.supervise(...).onFailure(...)*. Cette hiérarchie de supervision répond directement à l'exigence de l'énoncé sur la tolérance aux pannes dans les systèmes distribués critiques.

Les stratégies adoptées reflètent la criticité de chaque composant :

- *StorageActor* → *restart (max 3/10s)* : acteur critique. En cas d'exception, il repart à stock=0 plutôt que de faire tomber toute la colonie.
- *QueenActor* → *stop* : la mort de la reine est irréversible. Les acteurs qui la surveillent (*context.watch(queen)*) reçoivent le signal *Terminated* et s'arrêtent proprement.
- *ForagerAntActor*, *CarrierAntActor* → *restartWithBackoff(500ms–5s)* : une fourmi peut replanter (erreur Random, état incohérent) et reprendre en état idle sans menacer la colonie.

5.E) Pattern MessageAdapter (SimulationMonitor)

Le *SimulationMonitor* illustre le pattern MessageAdapter d'Akka Typed. Ce pattern est important car il découple le protocole interne d'un acteur (*MonitorCommand*) du protocole externe (*Command*) sans exposer les détails d'implémentation.

Concrètement, `context.messageAdapter[Command] { cmd => ... }` crée un *ActorRef[Command]* qui traduit automatiquement chaque *Command* reçu vers un *MonitorCommand*. Dans notre cas, les événements *FeedQueen*, *AntDied* et *Tick* sont traduits en *MonitorObserved(event: String)*. Ce pattern a des cas d'usage réels : traduction d'une API REST vers le protocole interne d'un acteur, réponses d'un service externe, etc.

6. Tests unitaires

6.A) Organisation

La version finale organise les tests en deux fichiers distincts dans *app/src/test/scala/* :

- *ActorSpec.scala* : tests des acteurs Akka avec Akka Actor TestKit (ScalaTest FlatSpec)
- *PetriNetSpec.scala* : tests du réseau de Pétri (ScalaTest FlatSpec, sans Akka)

Cette séparation améliore la maintenabilité : les tests Akka nécessitent une configuration TestKit (démarrage/arrêt d'un ActorSystem), tandis que les tests du réseau de Pétri sont de pures fonctions Scala sans état partagé.

6.B) Tests des acteurs (ActorSpec)

Acteur testé	Nb de tests	Cas couverts
StorageActor	7	Dépôt, fourniture, borne capacité, stock vide, ConsumeFood, demande > stock, séquences multiples
QueenActor	7	Tick, FeedQueen, Stop, SpawnAnt quota libre / quota atteint, AntDied fourrageuse / transporteuse
Invariants	12	Toutes les fonctions de vérification, cas limites (0, max, dépassement)
EggActor	3	Incubation Tick 1, éclosion après 2 Ticks, arrêt post-éclosion

6.C) Tests du réseau de Pétri (PetriNetSpec)

Groupe de tests	Description
1. Structure	18 places, noms uniques, poids P/T > 0
2. Marquage M_0	Correspondance exacte avec Simulation.scala
3. Franchissabilité depuis M_0	5 transitions activées, 6 bloquées
4. Franchissement fire	Vérification des marquages résultants après chaque transition

5. P-invariants sur M_0	Aucune violation attendue
6. Vérification analytique	$y^T \cdot C = 0$ pour PI1 (isPInvariant)
7. Exploration BFS	Max 5000 marquages
8. Détection deadlocks	Deadlocks uniquement si queen_dead
9. LTL1–LTL9	Toutes vérifiées sur l'espace d'états exploré
10. Scénarios bout en bout	Flux nominaux fourrageuse / transporteuse, mort de la reine, famine en 3 AntTick
11. Cycle de vie d'un œuf	egg_cycle1 $\rightarrow$ egg_cycle2 $\rightarrow$ forager/carrier (correction vs version précédente)
12. Correspondance Akka $\leftrightarrow$ Pétri	t_tick, t_carrier_deliver, t_search_food, etc.

6.D) Lien tests $\leftrightarrow$ vérification formelle

Les tests unitaires des acteurs Akka et la vérification formelle par réseau de Pétri sont complémentaires. Les tests unitaires garantissent le comportement correct sur des scénarios spécifiques (entrées précises, états bien choisis). La vérification formelle prouve les propriétés pour toutes les exécutions possibles, incluant les entrelacements rares que les tests ne couvriront jamais.

Par exemple, l'invariant LTL8 (deadlock $\rightarrow$ queen_dead) ne peut pas être prouvé par des tests unitaires car il nécessite l'exploration exhaustive de l'espace d'états. En revanche, un test peut rapidement détecter une régression sur le comportement nominal d'un acteur, là où la vérification formelle serait trop coûteuse à recalculer.

7. Simulation comparée : comportement réel vs modèle formel

7.A) Résultats de l'analyseur du réseau de Pétri

L'exécution de *sbt "runMain PetriNetMain"* produit un rapport complet de l'analyse formelle. Les résultats obtenus sur la branche finale sont les suivants :

Métrique	Valeur
Nombre de places	18
Nombre de transitions	31
Marquage M_0	{ carrier_idle=1, forager_idle=1, hunger_token=5, queen_alive=1 }
Transitions franchissables depuis M_0	5 (t_tick, t_lay_egg, t_search_food, t_request_food_empty, t_forager_starve_idle, t_carrier_starve_idle)
Marquages explorés (BFS, max 2000)	≥ 2000 (limite atteinte)
Deadlocks détectés	Uniquement sur marquages avec queen_dead=1
Violations P-invariants	0
Propriétés LTL1–LTL9	Toutes vérifiées

7.B) Observations de la simulation Akka

La simulation Akka (*sbt run*) produit un journal JSON Lines (*simulation_log.jsonl*) qui trace chaque événement : tick de la reine, dépôt et fourniture de nourriture, naissances et morts de fourmis. L'observation de plusieurs exécutions confirme le comportement prédit par le modèle formel :

- La faim de la reine oscille entre 0 et 10 sans jamais dépasser *MaxHunger* tant qu'au moins une transporteuse est active et que le stock n'est pas vide de façon prolongée. Cela valide LTL4.
- Les transporteuses livrent dès que le stock atteint 2 unités. Le mécanisme de priorité naturelle (livraison avant repos) garanti par l'ordonnancement des messages Akka est confirmé. Cela valide LTL6.
- Les deadlocks ne surviennent que lorsque la reine meurt (*hunger* > 10) : les acteurs qui la surveillent (*ForagerAntActor*, *CarrierAntActor*, *SimulationMonitor*) reçoivent le signal

Terminated et s'arrêtent proprement. Cela valide LTL8.

- Le cycle de reproduction est fonctionnel : les œufs pondus à chaque Tick progressent sur 2 cycles et éclosent en fourrageuse ou transporteuse selon un tirage aléatoire équiprobable. Cela valide TI4.

7.C) Limites du modèle formel et pistes d'amélioration

Le modèle formel présente deux limites principales par rapport au comportement réel de la simulation Akka :

1. Le temps et le non-déterminisme ne sont pas modélisés. Les schedulers Akka (`scheduleAtFixedRate`) créent un entrelacement temporel que le réseau de Pétri P/T pur ne peut pas capturer. Le réseau suppose que toute transition franchissable peut être choisie à tout moment, alors que dans Akka, les messages arriveront dans un ordre déterminé par les timers.

2. La quantité de nourriture collectée est approximée. La fourrageuse collecte entre 1 et 3 unités (aléatoire) dans Akka, mais la transition `t_search_food` utilise un poids fixe de 2 (valeur moyenne) dans le réseau de Pétri. Cela peut légèrement surestimer ou sous-estimer la production de nourriture selon les runs.

Ces limitations sont inhérentes aux réseaux de Pétri P/T purs. Des extensions comme les réseaux de Pétri temporels (Time Petri Nets) ou colorés (Coloured Petri Nets) permettraient une modélisation plus fidèle, mais au prix d'une complexité d'analyse nettement supérieure.

8. Sources bibliographiques

8.A) Réseaux de Pétri — fondements théoriques

- Petri, C.A. (1962). Kommunikation mit Automaten. PhD thesis, Universität Hamburg. (Source originale de la formalisation des réseaux de Pétri)
- Murata, T. (1989). Petri nets: Properties, analysis and applications. Proceedings of the IEEE, 77(4), 541–580. (Référence standard pour les propriétés structurelles (bornitude, vivacité, P/T-invariants))
- Reisig, W. (2013). Understanding Petri Nets: Modeling Techniques, Analysis Methods, Case Studies. Springer. (Introduction complète avec études de cas sur les systèmes distribués)

8.B) Vérification formelle et model checking

- Clarke, E.M., Grumberg, O., Peled, D. (2000). Model Checking. MIT Press. (Référence principale pour le model checking et la logique LTL)
- Baier, C., Katoen, J.-P. (2008). Principles of Model Checking. MIT Press. (Traitement rigoureux de LTL, CTL et les algorithmes d'exploration d'espace d'états)
- Holzmann, G.J. (2004). The SPIN Model Checker. Addison-Wesley. (Implémentation pratique du model checking, utile pour comprendre les limites des approches BFS)

8.C) Modèle d'acteurs et Akka

- Hewitt, C., Bishop, P., Steiger, R. (1973). A universal modular actor formalism for artificial intelligence. IJCAI 1973. (Article fondateur du modèle d'acteurs)
- Vernon, V. (2016). Reactive Messaging Patterns with the Actor Model. Addison-Wesley. (Bonnes pratiques Akka pour les systèmes distribués critiques)
- Documentation officielle Akka Typed 2.8.x : <https://doc.akka.io/docs/akka/current/> (Référence pour la supervision, les FSM typées, et le pattern MessageAdapter)

8.D) Scala fonctionnel

- Odersky, M., Spoon, L., Venners, B. (2021). Programming in Scala, 5th ed. Artima Press. (Référence principale pour Scala 2)
- Chiusano, P., Bjarnason, R. (2014). Functional Programming in Scala. Manning Publications. (Approche fonctionnelle pure applicable à la modélisation des acteurs)